FIT3155: Week 7 Lab Sheet

(Questions are based on Week 6 topic dealing with semi-numerical algorithms.)

Objectives: Develop an understanding of Karatsuba big integer multiplication, modular exponentiation, and the Miller-Rabin primality test.

Conceptual exercises

☑ 1. Suppose $U = (u_1 u_2 \dots u_d)_\beta$ is a d-digit integer represented in base $\beta \geq 2$, where each digit $u_i \in \{0, 1, \dots, \beta - 1\}$. Write a formula that allows you to interpret this positional notation in base β as an integer in $\mathbb{Z}$.

☑ 2. Python uses arbitrary-precision integers by default. In what base are integers represented internally in Python (note: the answer is not base 2)?

☑ 3. In the lecture, Karatsuba's algorithm was derived for multiplying two $2d$-bit integers. Generalise this decomposition to obtain a similar divide-and-conquer structure for Karatsuba's algorithm for multiplying two $2d$-digit integers in an arbitrary base b. (After this, reason about which base this algorithm would be better implemented in when done on a computer architecture with a word size of w, where multiplication of w-bit integers is supported as a single hardware operation.)

☑ 4. Prove that the divide-and-conquer approach for integer multiplication (irrespective of the base) has time complexity of $O(d^{\log_2 3})$ by explicitly solving the following recurrence relation (without using the Master Theorem):

$$T(2d) = 3T(d) + cd,$$

where c is a constant.

☑ 5. Consider a binary representation of a number $u = (u_1 u_2 u_3 \dots u_d)_2$. Can this representation be converted to an integer in $\mathbb{Z}$ by processing the bits from the most significant bit to the least significant bit? If yes, explain how; if no, explain why not.

☑ 6. In lecture, you learnt an algorithm for modular exponentiation $a^b \bmod n$ using repeated squaring, which processes the binary representation of the exponent b from the least significant bit to the most significant bit order. Linking to the above question, is it possible to perform a similar approach for modular exponentiation while instead processing

the bits from the most significant bit to the least significant bit order? If yes, write its pseudocode.

☑ 7. Compute 7^{330} mod 13 by hand using the repeated squaring method.

☑ 8. Let n be an odd integer. For the bases $a = 1$ and $a = n - 1$, show that $a^{n-1} \equiv 1 \bmod n$.

☐ 9. Show that any integer $z > 0$ can be uniquely written in the form $z = 2^s t$, where $s \geq 0$ and t is odd.

10. Let $y^2 \equiv 1 \pmod{n}$.

☑ (a) Determine all possible solutions for $y \pmod{n}$ when n is prime.

☑ (b) Can additional solutions exist when n is composite? Justify your answer.

☐ 11. Work out all the steps corresponding to one iteration of the Miller–Rabin primality test on $n = 21$ using the base (say) $a = 13$.

Implementation exercises

(A message of caution: 'Vibe coding' with GenAI will hinder your learning of algorithms and data structures. The degree (and this unit) you have enrolled into is a specialist degree (unit). By definition it expects one to demonstrate specialist understanding of topics. This understanding comes from making a concerted effort, via thinking, implementing, debugging etc.)

☑ 1. Implement modular exponentiation using the method of repeated squaring.

☑ 2. Implement Miller-Rabin's randomized method of primality testing. Generate the the first 10,000 primes using this method.

☑ 3. Implement the divide-conquer approach for multiplication of two integers.

see karatsuba.py

```
-=0=-
 END
-=0=-
```